

Elaborated Claude Documentation

Prompt

Project: Automated: A Multi-Agentic Trading System

Authors: Abdul Qadeer Khan & Ahmad Abdullah

Created: May 2026

Operational Instructions:

Copy the entire prompt block below and paste it directly into Claude (or any LLM) to instantly generate an extremely detailed, 11-page technical documentation manual complete with diagrams and code blocks for your final BSCS thesis!

THE PROMPT TO COPY & PASTE TO CLAUDE:

You are a Principal Software Architect, Elite Technical Writer, and Senior BSCS Thesis Advisor. Your goal is to write a comprehensive, publication-grade, 11-page tutorial-based Technical Documentation Manual for our newly deployed project: "Automated: A Multi-Agentic Trading System".

This system is built using Python FastAPI, SQLAlchemy ORM, Neon PostgreSQL, OpenAI Agents SDK, Langfuse Tracing, yfinance, and custom Vanilla CSS/JS. It is deployed live on Render's Free tier.

🍷 DOCUMENTATION LAYOUT STRUCTURE (Targeting 11 Distinct Chapters/Pages)

You must expand each page/chapter with extreme technical depth, complete configuration files, production-grade code snippets, architecture rationale, and step-by-step tutorial tasks. Do not write generic summaries. Inject precise code variables, files, and architectural paradigms.

📄 Page 1: Title, Executive Summary & Project Paradigm

- * Act as an institutional manual cover. Include formal project title, creators (Abdul Qadeer Khan & Ahmad Abdullah), and course details.
- * Provide a comprehensive executive summary of the system.
- * Explain the paradigm shift from single-prompt monolithic LLMs to Decoupled, Multi-Agent Asynchronous Orchestration. Detail why separation of concerns prevents context contamination and preserves prompt token budgets.

📄 Page 2: System Architecture & Decoupled Orchestration Flow

- * Provide a comprehensive deep dive into the information lifecycle.
- * ****INCORPORATE DIAGRAM 1**** (Multi-Agent Decoupled Orchestration Flow) directly as Mermaid code.
- * Explain each agent's execution sandbox, why the Market Analyst holds ZERO execution authority, and how the system dynamically interfaces with `yfinance` to pass structured schemas.

📄 Page 3: Agent Registry & Persona Specifications

- * Detail the system rules and operational laws governing the three active nodes:
 1. Master Trading Desk Router (Orchestrator)
 2. Intraday & Swing Market Analyst (Signal Synthesizer)
 3. Risk Manager Agent (Compliance Auditor)
- * Provide the exact Pydantic output schemas (`MarketAnalysis`, `RiskEvaluation`, `FinalTradingDecision`) from `schemas.py` and explain their mathematical attributes.

📄 Page 4: Secure Operator Authentication & Session Controls

- * Detail the database registration and login architecture.
- * Explain the cryptographic password hashing (bcrypt) and Pydantic validator models for name, internationalized mobile phone parsing (`^(03|\+923)\d{9}\$`), and the loosened 6-character flexible password complexity standard.
- * Explain the frontend session locking mechanisms that decrypt and authorize dashboard access only upon successful database session verification.

📄 Page 5: Programmatic Transaction-Isolated Startup Migration Sequence

- * Document the critical database columns discrepancy bug and how it was resolved.
- * ****INCORPORATE DIAGRAM 2**** (Programmatic Transaction-Isolated Startup Sequence) directly as Mermaid code.
- * Provide the exact async startup Python block from `main.py` that separates column renaming and metadata creation into two distinct connection/transaction sessions, preventing the psycopg `InFailedSqlTransaction` transaction abort crashes.

📄 Page 6: Dynamic Strategy Data Matrix

- * Discuss the dynamic lookback parameters used to balance historical depth with LLM context window conservation.
- * Include a markdown table documenting the active intervals, periods, indicators considered (EMA Crossovers, MACD Histograms, RSI Overbought thresholds), and calculated exit boundary metrics for all 4 strategy classes (Scalp, Intraday Swing, Medium Swing, Macro Position).

📄 Page 7: Interactive SVG Candlestick Charting Engine

- * Document the client-side plotting algorithm that draws real-time beating wicks and bodies inside the SVG canvas.
- * ****INCORPORATE DIAGRAM 3**** (SVG Candlestick Rendering & Scaling) directly as

Mermaid code.

- * Explain the math behind SVG coordinate systems (where Y=0 is at the top), how min/max prices are scaled with 8% padding margins, how Take Profit / Stop Loss boundary vectors are mapped directly over the candles, and how the 10-second polling interval updates the UI in real time.

📄 Page 8: Telemetry, Observability & Database Logging

- * Explain the Neon PostgreSQL persistent run registry, detailing what fields are saved in the `trading\_history` table.
- * Document the Langfuse Tracing implementation: trace initializations, nested execution spans for each agent call, and database payload instrumentation, showing why this meets institutional observability requirements.

📄 Page 9: Step-by-Step Installation, uv Environment Management & Setup

- * Provide a comprehensive tutorial for a developer setting up the workspace from scratch.
- * Include commands to clone, initialize the virtual environment with `uv`, install exact dependencies using the autogenerated `requirements.txt`, configure `.env` variables, and spin up the local FastAPI development server.

📄 Page 10: Render Cloud Deployment Playbook

- * Provide a detailed, step-by-step guide to deploying the system on Render for free.
- * Document Git pushes, Web Service configurations, Build Commands (`pip install -r requirements.txt`), Start Commands (`uvicorn main:app --host 0.0.0.0 --port \$PORT`), Environment Variables configuration, and a technical discussion on how to mitigate Free-tier inactivity cold starts.

📄 Page 11: Troubleshooting, Operational Best Practices & Thesis QA Guide

- * Detail a troubleshooting runbook for connection timeouts, invalid tokens, or database lockouts.

* Provide a formal "Thesis Defense Panel Q&A Guide" with 5 common technical questions a computer science thesis panel will ask about this multi-agent system and the exact, highly technical answers to defend the implementation.

📊 READY-TO-USE MERMAID CODES (Include these inside the generated chapters)

🌀 DIAGRAM 1: Multi-Agent Decoupled Orchestration Flow

```mermaid

graph TD

User([Operator / User]) -->|1. Authentication & Parameters| Router[Master Desk Router]

Router -->|2. Ingest Inputs & Init Trace| Telemetry[Langfuse Global Trace]

Router -->|3. Fetch Historical Candlesticks| YFinance[(yfinance Data Extraction)]

YFinance -->|4. Parse Indicators: EMA, RSI, MACD| Analyst[Market Analyst Agent]

Analyst -->|5. High-Fidelity Signal Synthesis| Risk[Risk Manager Agent]

Risk -->|6. Compliance Gate: APPROVED/REJECTED| Override{Override Check}

Override -->|If APPROVED| Alloc[Deploy Position Sizing Budget]

Override -->|If REJECTED| Force[Override to HOLD / 0% Allocation]

Alloc -->|7. Persist Result| DB[(Neon PostgreSQL DB)]

Force -->|7. Persist Result| DB

DB -->|8. Render Dashboard Metrics & Chart wicks| UI[Interactive Front-End Web UI]

```

🌈 DIAGRAM 2: Programmatic Transaction-Isolated Startup Sequence

```
```mermaid
```

```
sequenceDiagram
```

```
 autonumber
```

```
 participant FastAPI as FastAPI App Startup
```

```
 participant DB as Neon PostgreSQL Engine
```

```
 participant Session as Connection Session
```

```
 Note over FastAPI, DB: Isolated Transaction Block 1 (RENAME Column)
```

```
 FastAPI->>DB: Open connection: engine.connect()
```

```
 FastAPI->>Session: Begin Transaction: conn.begin()
```

```
 FastAPI->>DB: Execute: ALTER TABLE system_users RENAME COLUMN mobile_number TO
verified_phone;
```

```
 alt Column/Table Already Renamed or Clean DB
```

```
 DB-->>FastAPI: Raise error (Exception)
```

```
 FastAPI->>Session: Trigger: transaction.rollback()
```

```
 Note over Session: Rollback clears aborted transaction state
```

```
 else First-time Migration Succeeds
```

```
 DB-->>FastAPI: Success
```

```
 FastAPI->>Session: Trigger: transaction.commit()
```

```
 end
```

```
 FastAPI->>-DB: Close connection Session
```

```
 Note over FastAPI, DB: Isolated Transaction Block 2 (Create Table Metadata)
```

```
 FastAPI->>DB: Open transaction: engine.begin()
```

```
FastAPI->DB: Execute: Base.metadata.create_all
```

```
DB-->FastAPI: Create missing tables cleanly
```

```
FastAPI->-DB: Close transaction & engine.dispose()
```

```
Note over FastAPI: Startup Completed Successfully!
```

```
...
```

```
🐙 DIAGRAM 3: SVG Candlestick Rendering & Scaling
```

```
```mermaid
```

```
graph TD
```

```
    Data[Raw yfinance Candle Close/Open/High/Low] --> Bounds[Find Min/Max Prices  
in Candle Set]
```

```
    Bounds --> Padding[Apply 8% Margins padding to Max/Min]
```

```
    Padding --> Scale[Calculate Price Range: Max - Min]
```

```
    Scale --> Ratio[Map to SVG viewBox: 400 width x 200 height]
```

```
    Ratio --> WickY[Y-Coordinate = 200 - Price-Min / PriceRange * 200]
```

```
    WickY --> Wicks[Draw Wick Line: High Y to Low Y]
```

```
    WickY --> Body[Draw Body Rectangle: Min Open/Close Y, Height = Abs Open-Close]
```

```
    Body --> Color{Bullish vs Bearish}
```

```
    Color -->|Bullish Close >= Open| Green[Render Emerald Green #10B981]
```

```
    Color -->|Bearish Close < Open| Orange[Render Alert Orange #F97316]
```

```
    Green --> Draw[Draw inside SVG Canvas]
```

```
    Orange --> Draw
```

```
...
```

```
---
```

📝 WRITING STYLE & TONE GUIDELINES

- * Write with formal, authoritative academic prose mixed with rigorous software engineering vocabulary.
- * Structure each page with precise Markdown headers, subheaders, bullet points, and codeblocks.
- * Do not truncate or use placeholders. Generate the complete 11-page manuscript in full!